

AI-Patch-Gate auf Windows

m läuft in WSL, im Linux-Dateisystem der Distribution. Der Installer ist eine Windows-Oberfläche und schreibt nur in den angegebenen Quellbaum. Die Windows-Benutzerumgebung und `~/ .bashrc` lässt er unangetastet. Der Compiler bleibt die Prüfung. Ein Urteil, eine abgelehnte Frage und ein früher fehlgeschlagener Stand sperren den nächsten m nicht.

Zwei Betriebsarten, wie unter Linux:

- **blockierend, Vorabtest** ist vorausgewählt. Frage und Urteil erscheinen, bevor Ninja startet. j, ja, y oder yes lässt den Build zu, auch wenn das Modell `likely_fail` sagt, und dieses Ja wird gemerkt. Die Eingabetaste, n oder alles andere stoppt nur diesen Lauf und wird nicht gemerkt. Der nächste m fragt wieder. Ein Stand, der schon einmal fehlgeschlagen ist, startet trotzdem.
- **parallel, nur Hinweis** startet den Build sofort und bricht ihn von selbst nicht ab. Nach dem Build stehen die Hinweise noch einmal zwischen `----- ai-patch-gate -----`. Wird der Build fertig, während die Anfrage noch läuft, bricht der Gate die Anfrage ab. Der nächste m prüft erneut. Es gibt keine 24-Stunden-Pause. Strg-C und der Stop-Button von VS Code beenden den Build. Unter Windows öffnet ein Verdacht zusätzlich ein Fenster, siehe unten.

Ohne Terminal wird die Frage gedruckt und der Build startet. `AAOS_AI_GATE=stop` ist die einzige harte Weigerung, und nur bei `likely_fail`. Die vom Installer geschriebene Datei setzt `AAOS_AI_GATE=on`.

Die Rückfrage `So beabsichtigt? [j/N]` merkt sich nur ein Ja, in `out/.aaos-ai-gate/answers` in der Distribution. `m clean` löscht nur dieses Verzeichnis. `~/ .cache/aaos-ai-gate` in WSL ist der Diff-Cache und bleibt. Ein Abbruch oder ein Rückgabewert ab 128 wird nicht als Compile-Fehler gemerkt.

Diff, der oberste Stash und die Betreffzeilen der letzten Commits gehen an den Anbieter. Claude mit Token an `api.anthropic.com`, Antigravity mit Token an Google, Copilot und jedes Abonnement an die jeweilige Kommandozeile, eine eigene URL an diese URL. Die eigenen Gate-Dateien schickt er nicht mit. Fehlt `jq` in der Distribution, sagt er das und der Build startet trotzdem. Ein Baum ohne Diff, den er schon kennt, läuft nicht erneut über alle Repo-Projekte.

Dialog im parallelen Modus

Die Skizze dieses Fensters steht in `aaos-ai-patch-gate-windows.pdf`. Die Nummern dort sind dieselben wie in der Erklärung unter der Zeichnung.

Das Fenster erscheint nur unter Windows und nur im parallelen Modus, wenn das Modell etwas Konkretes meldet: eine Rückfrage, einen wahrscheinlichen Compile-Bruch, eine absehbare Laufzeitfolge oder einen verfehlten Effekt. Der Build läuft dabei weiter. Ein unauffälliger Diff öffnet nichts.

Oben steht der Verdacht in Klartext. Darunter kann man demselben Anbieter eine Rückfrage schicken. Die Eingabetaste und der Knopf Fragen tun dasselbe. Ist kein Anbieter hinterlegt, bleibt Fragen aus. Schließen legt das Fenster weg. Der nächste Verdacht öffnet es wieder.

Nicht mehr anzeigen gilt beim Schließen. Es legt `.aaos-ai-gate.dialog-off` in die Wurzel des Quellbaums. `AAOS_AI_GATE_DIALOG=off` schreibt es in die Konfiguration nur, wenn eine Konfiguration im Baum existiert. Beides entfernen, dann ist der Dialog wieder da. Auf blockierend umstellen schreibt den Modus in `.aaos-ai-gate.conf` und gilt ab dem nächsten m. Dieser Build läuft zu Ende. Auch im blockierenden Modus startet j den Build, wenn das Modell einen Compile-Bruch erwartet. Steht `AAOS_AI_GATE_MODE` in der Shell, gewinnt die Shell gegenüber der Datei. Jedes Element im Fenster hat einen Tooltip.

Installer

Die Skizze des Installers steht in derselben PDF. Jedes Feld, beide Kästen, Installieren und das Ergebnisfeld sind dort nummeriert und direkt unter der Zeichnung erklärt. Entfernen steht als Umriss-Knopf neben Installieren und ist bei Punkt 10 erklärt.

In PowerShell, im Verzeichnis `aaos-ai-gate`:

```
powershell -NoProfile -STA -ExecutionPolicy Bypass -File .\install-windows.ps1
```

Das Fenster fragt nach dem Quellbaum, dem Modus, dem Anbieter und der Anmeldung. Anbieter sind Claude, Microsoft Copilot und Antigravity. Beim Öffnen und nachdem der Quellbaum feststeht, sucht es in der Distribution nach `claudio`, `copilot` und `agy`. Der Eintrag zeigt „installiert“ oder „nicht gefunden“, darunter steht, was gefunden wurde. Vorausgewählt ist die erste gefundene Kommandozeile, in der Reihenfolge Claude, Microsoft Copilot, Antigravity. Liegt der Baum unter `\\wsl$`, gilt diese Distribution, sonst die Standard-Distribution. Ist keine Kommandozeile da, bleibt Claude vorausgewählt. Die Anmeldung ist das Abonnement, also die schon vorhandene Anmeldung in der Distribution, oder ein API-Token. Bei Abonnement ist das Token-Feld grau und wird nicht gebraucht. Eine Endpoint-URL zeigt das Fenster erst, wenn man „Abweichende Verbindung angeben“ ankreuzt. Modellname und URL dürfen dann leer bleiben. Eine eingetragene URL braucht ein API-Token. Fehlt die Kommandozeile zum gewählten Abonnement, steht das im Ergebnisfeld, die Installation läuft trotzdem. Microsoft Copilot braucht `copilot` auch mit API-Token. Der Baum ist am besten ein `\\wsl$<Distribution>\...-Pfad`, zum Beispiel `\\wsl$Ubuntu\home\<name>\aosp`. Durchsuchen öffnet die Ordnerauswahl.

Vor dem Kopieren prüft der Installer `jq` in der Distribution. Auf dem Token- oder URL-Weg prüft er dort auch `curl`. Fehlt eines, oder ist WSL nicht erreichbar, ändert er den Baum nicht. Zum Beispiel in der Distribution `sudo apt install jq` bzw. `sudo apt install curl`.

Zwei Kästen darunter:

- **ccache in ~/.bashrc.** Der Kasten ist nur wählbar, wenn „Umgebung selbst eintragen“ an ist. Die Zeilen stehen dann im Ergebnisfeld für `~/.bashrc` der Distribution. In die Gate-Datei kommen sie nicht. Die liest nur der Hook in `build/soong/bin/m.mm, mmm, mma` und `mma` sind eigene Skripte, `make` aus `envsetup.sh` ruft `soong_ui.bash` direkt. Ein Wechsel änderte sonst `CC_WRAPPER`, und Soong und Kati erzeugten neu. Nach dem Entfernen des Gates wäre `ccache` still weg und der nächste Build kalt. Verzeichnis und Größe sind Linux-Pfade, Voreinstellung `~/.cache/ccache` und 100G. `~`, `~/`, `$HOME` und `${HOME}` werden in der Distribution zum Home-Verzeichnis. `CCACHE_EXEC` ist der Pfad, den `command -v ccache` dort liefert, kein festes `/usr/bin/ccache`. Fehlt das Paket, setzt er `USE_CCACHE` nicht und nennt `sudo apt-get install -y ccache`. Die Größe merkt sich `ccache` über `-M.CCACHE_MAXSIZE` schreibt er nicht. AOSP setzt `compilercheck` und `Sloppiness` in `build/make/core/ccache.mk` selbst. `ccache` trifft nur C und C++.
- **Umgebung selbst eintragen.** Dann entsteht keine `.aosp-ai-gate.conf`. Das Ergebnisfeld zeigt die Zeilen für `~/.bashrc` in WSL und, als zweite Variante, die PowerShell-Befehle plus `WLENV`. Die muss man selbst ausführen. Einen vorhandenen `WLENV` anhängen, nicht ersetzen. Die angehängte Zeile ist `if ($existing) { $gate = "${existing}:$gate" }`. `/u` reicht den Wert nach WSL durch und übersetzt ihn nicht als Pfad.

Ohne diesen Kasten schreibt der Installer `$TOP/.aosp-ai-gate.conf`. Auf einem `\\wsl$`-Baum setzt er über WSL die Modus-Bits 755 für das Skript und 600 für die Konfiguration. Auf einem Windows-Laufwerk gilt `chmod` nicht. Die Konfiguration bekommt dort die Rechte über `icacls`, und das Ergebnisfeld sagt, dass der Unix-Modus 600 auf diesem Laufwerk nicht gilt. Die Datei setzt nur `AAOS_AI_GATE` und Namen mit `AAOS_AI_GATE_`. `ccache`-Zeilen, `PATH` und `LD_PRELOAD` darin ignoriert der Gate. `m` ruft den Gate nur auf, wenn diese Datei existiert oder `AAOS_AI_GATE` gesetzt ist. Shell-Variablen gewinnen gegenüber der Datei.

Kopiert werden `build/soong/bin/ai-patch-gate.sh` und `aosp-ai-gate-dialog.ps1`.

`build/soong/bin/m` bekommt den Aufruf. Fehlt dort die Zeile `_wrap_build`

`"$TOP/build/soong/soong_ui.bash"`, bleiben die Skripte im Baum und das Ergebnisfeld zeigt den Block zum Einsetzen. Der Aufruf läuft über `_wrap_build`, damit die Zeile „#### build completed successfully ...“ erhalten bleibt. Ein abgelehnter Gate (Exit 2) erscheint als „failed to build some targets“. Die Skripte kommen in die Git-Ausschlussliste von `build/soong/bin/m` wird `skip-worktree`, damit `repo status` den Hook nicht zeigt. Rückgängig in WSL: `git -C build/soong update-index --no-skip-worktree bin/m`.

Entfernen nimmt Hook, Skripte, Konfiguration und die Ausschlusslisten aus dem Baum. Der Diff-Cache und `out/` bleiben. Im Fenster ist das der Knopf neben Installieren. Ohne Fenster, mit gesetztem `AAOS_INSTALL_TREE`:

```
$env:AAOS_INSTALL_TREE = '\\wsl$Ubuntu\home\<name>\aosp'
```

```
powershell -NoProfile -STA -ExecutionPolicy Bypass -File .\install-windows.ps1 -Uninstall
```

Von Hand

Das Diff `aaos-ai-patch-gate.diff` liegt neben diesem Text. Die Installer sind nicht darin. In WSL, von der Wurzel des Checkouts:

```
patch -p1 < /mnt/c/Users/<name>/devel/deliverables/aaos-ai-gate/aaos-ai-patch-gate.diff
chmod +x build/soong/bin/ai-patch-gate.sh
```

`git apply` setzt das Executable-Bit selbst. Mit dem Diff kommt auch `build/soong/bin/aaos-ai-gate-dialog.ps1`. Ohne diese Datei bleibt der Hinweis im Log. Danach in `~/.bashrc` der Distribution:

```
export AAOS_AI_GATE=on
export AAOS_AI_GATE_MODE=blocking
export AAOS_AI_GATE_PROVIDER=claude
export AAOS_AI_GATE_AUTH=subscription
```

`AAOS_AI_GATE_AUTH=token` braucht zusätzlich `AAOS_AI_GATE_TOKEN`. Copilot heißt `copilot`, Antigravity `antigravity`. Dieselben Namen kann Windows per Benutzerumgebung und `WSLENV` durchreichen. `AAOS_AI_GATE_CACHE` nur als Linux-Pfad in WSL setzen.

Variable	Bedeutung
<code>AAOS_AI_GATE</code>	<code>on</code> schaltet den Gate ein. <code>stop</code> ist die einzige harte Weigerung, und nur bei <code>likely_fail</code> .
<code>AAOS_AI_GATE_MODE</code>	<code>blocking</code> für den Vorabtest, <code>parallel</code> für nur Hinweis. Ohne Setzen gilt <code>blocking</code> .
<code>AAOS_AI_GATE_DIALOG</code>	<code>off</code> unterdrückt das Windows-Fenster. Dieselbe Wirkung hat die Datei <code>.aaos-ai-gate.dialog-off</code> im Quellbaum.
<code>AAOS_AI_GATE_PROVIDER</code>	<code>claude</code> , <code>copilot</code> oder <code>antigravity</code> .
<code>AAOS_AI_GATE_AUTH</code>	<code>subscription</code> für die vorhandene Anmeldung, <code>token</code> für ein API-Token.
<code>AAOS_AI_GATE_TOKEN</code>	Nur bei <code>token</code> , oder wenn eine eigene URL gesetzt ist.
<code>AAOS_AI_GATE_URL</code>	Nur auf ausdrücklichen Wunsch. Ersetzt dann den Anbieter und braucht ein Token.
<code>AAOS_AI_GATE_MODEL</code>	Nur setzen, wenn ein bestimmtes Modell gewünscht ist. Sonst die Voreinstellung des Anbieters. Bei einer eigenen URL ohne Modellname gilt <code>default</code> .
<code>AAOS_AI_GATE_BASE</code>	Git-Revision. Voreinstellung <code>HEAD</code> .
<code>AAOS_AI_GATE_MAX_BYTES</code>	Obergrenze des neu geschickten Diffs. Voreinstellung <code>80000</code> .
<code>USE_CCACHE</code> , <code>CCACHE_EXEC</code> , <code>CCACHE_DIR</code>	Nur in der Shell der Distribution, zum Beispiel <code>~/.bashrc</code> . Die Gate-Datei liest sie nicht. Sonst gilt <code>ccache</code> nur für <code>m</code> .

Neues WSL-Terminal, dann:

```
source build/envsetup.sh
lunch sdk_car_x86_64-userdebug
m
```

In VS Code ist der Gate die WSL-Shell-Task, die `m` aufruft. Der Stop-Button bricht den Build ab. Android Studio sieht ihn, wenn `m` in seinem Terminal läuft.

```
{
  "label": "AAOS m",
  "type": "shell",
  "command": "source build/envsetup.sh && lunch sdk_car_x86_64-userdebug && m",
  "options": { "cwd": "${workspaceFolder}" },
  "problemMatcher": []
}
```